Mathematical Modelling of Population Harvesting under a Strong Allee Effect

Equilibria, Stability, Extinction and PINN Analysis

Raghav Singhal Ashwath Chandramouli Devam Bhavsar
Mehta Prem Rohan Agrawal

BITS Pilani, K K Birla Goa Campus

Mathematical Modelling, Semester I, 2026–27

November 2026

Abstract

We study population harvesting under a strong Allee effect, where a population below a certain size may continue to decline even without harvesting. Two harvesting methods are considered: constant harvesting (Model A) and proportional harvesting (Model B). We derive the equilibrium points, analyse their stability, and calculate the critical harvesting rates beyond which no positive equilibrium exists. We also examine how the initial population affects survival, calculate extinction times, and study delayed collapse near the critical harvesting rate. The effect of changing harvesting rates and restocking on population recovery is also investigated. Numerical results are checked using independent integration methods. We then apply Physics-Informed Neural Networks (PINNs) to solve the population equations and estimate unknown parameters from synthetic observations. For the forward problem, the PINN achieves relative L^2 errors ranging from 2.9×10^{-7} to 4.1×10^{-3}, although conventional ODE solvers are considerably faster. For the inverse problem, the PINN recovers all three fitted parameters within 5% of their true values in 10 out of 18 noise-free runs, compared with 18 out of 18 for ODE least-squares fitting. We also study the effect of measurement noise and limitations of parameter estimation, including nonidentifiability in Model B when all four physical parameters are unknown. All experiments use synthetic data, and the results describe the mathematical models rather than any particular real-world population.

1 Introduction and objectives

1.1 Motivation

Many harvested populations, fish stocks in particular, have collapsed and then failed to come back even after fishing was reduced [7]. One reason is the *Allee effect*: when a population is small, its members do worse, for example because they find mates less easily, defend themselves less well against predators or lose the benefits of living in groups [15, 3]. If the effect is strong, there is a critical population size below which natural growth becomes negative. In the model studied here, we denote this Allee threshold by A [4, 3].

Combine such a threshold with fishing and several practical questions follow. How much can be caught each year without risking collapse? How close to collapse is a given catch? Does it matter whether we catch a fixed amount or a fixed fraction of whatever is there? And if a stock has collapsed, does lowering the catch bring it back?

This project answers these questions for two simple fishing rules, using mathematics first and computation second. We then ask a separate, more modern question: can a physics-informed neural network (PINN) [11] solve the same equations, and can it recover unknown model parameters from noisy data? The answer to the second part turns out to be more mixed, and we report the failures as carefully as the successes.

1.2 Objectives

The report has five parts, each with a different goal.

1. **Foundations (Section 2).** Find the steady populations of a stock fished with a constant quota, decide which are stable, and locate the largest quota that still allows a steady population.

2. **Proportional harvesting (Section 3).** Do the same for a fixed fishing effort, compare the two rules fairly, and test whether lowering the catch undoes a collapse.

3. **Collapse (Section 4).** Compute the critical catch, the starting populations that lead to extinction, the time extinction takes, and why collapse can seem to arrive late.

4. **Forward PINN (Section 5).** Train a small neural network to solve the equations without being shown any solution, and compare it with a standard numerical solver.

5. **Inverse PINN (Section 6).** Estimate the unknown model parameters from noisy synthetic observations with a PINN and with ordinary curve fitting, and find out when the parameters can and cannot be recovered.

1.3 Assumptions, model and notation

We study one closed population that mixes well and has no age structure, no migration, no other species and no random fluctuations. Its size is $N(t) \geq 0$ at time t. All parameters are constant. The three growth laws we compare are

$$\text{Logistic with constant catch:} \quad \frac{dN}{dt} = rN\Big(1 - \frac{N}{K}\Big) - H, \tag{1.1}$$

$$\text{Model A (strong Allee, constant catch):} \quad \frac{dN}{dt} = rN\Big(1 - \frac{N}{K}\Big)\Big(\frac{N}{A} - 1\Big) - H, \tag{1.2}$$

$$\text{Model B (strong Allee, proportional effort):} \quad \frac{dN}{dt} = rN\Big(1 - \frac{N}{K}\Big)\Big(\frac{N}{A} - 1\Big) - hN. \tag{1.3}$$

The extra factor $(N/A - 1)$ is the strong Allee effect: growth is negative whenever $0 < N < A$. Table 1.1 lists the symbols.

Two conventions matter throughout. First, a fixed quota cannot be taken from an extinct stock, so a Model A or logistic run stops when the population reaches zero (we call this *absorbing extinction*; continuing the equation would give negative populations). Second, under Model B the catch is proportional to the population, so a positive population never reaches exactly zero in finite time; it only decays towards it.

Table 1.1: Symbols and units. A dimensionless version of each equation is used for all calculations (Section 2).

Symbol	**Meaning**	**Unit**
$N,\ t$	population size, time	tonnes (or number), years
r	intrinsic growth rate	1/time
K	carrying capacity	same as N
A	Allee threshold ($0 < A < K$)	same as N
H	constant catch (quota)	N/time
h	fishing effort, a fraction removed per unit time	1/time
$x = N/K,\ \tau = rt$	scaled population and scaled time	none
$a = A/K,\ q = H/(rK),\ e = h/r$	scaled threshold, quota and effort	none

About the numbers. Every example in this report is *synthetic*: we choose the parameters and generate the data by solving the equations. Nothing was measured from a real population, so our results describe the models rather than any particular species.

Sections 2 and 3 use $r = 0.5$, $K = 1000$ and $A = 200$ for their numerical examples. Section 4 and the PINN experiments use $r = 0.7$, $K = 100$ and $A = 20$. Both parameter sets have $a = A/K = 0.2$, so their dimensionless critical harvesting rates agree.

Use of AI. AI tools assisted with parts of the mathematical explanations, implementation, testing and report preparation. The platforms and their roles are acknowledged in Section 8. The numerical checks are described in the relevant sections, while the larger PINN experiments are reported using their recorded results rather than being rerun for this report.

2 Foundations and Model A: steady populations under a fixed catch

Question. Suppose the same amount is caught every year. At what population sizes can the stock hold steady, which of these are safe, and how large can the catch be before collapse becomes certain?

2.1 Scaling

The model has four parameters, but only two combinations of them actually matter. If we measure the population as a fraction of capacity, $x = N/K$, and time in units of $1/r$, $\tau = rt$, the chain rule gives $dN/dt = rK\,dx/d\tau$. Dividing Eq. (1.2) by rK then leaves

$$\frac{dx}{d\tau} = f(x) = G(x) - q, \qquad G(x) = x(1-x)\Big(\frac{x}{a} - 1\Big), \tag{2.1}$$

where $a = A/K$ is the Allee threshold as a fraction of capacity and $q = H/(rK)$ is the scaled catch. In the same way the logistic model becomes $dx/d\tau = x(1-x) - q$, and Model B becomes $dx/d\tau = G(x) - ex$ with $e = h/r$. Any result can be turned back into real units using $N = Kx$, $t = \tau/r$ and $H = rKq$. All examples in this section use $a = 0.2$.

2.2 Steady states and stability

A steady state, or equilibrium, is a population x_e at which growth and catch cancel out, so that $f(x_e) = 0$ and nothing changes. To decide whether it is stable, we look at the sign of f just on either side of it [16]. If f is positive just below x_e and negative just above, a population nudged slightly away is pulled back, and the state is *stable*. If the signs are the other way round, small nudges grow and the state is *unstable*. If f has the same sign on both sides, the state attracts from one side and repels on the other, and we call it *semistable*. Wherever $f'(x_e) \neq 0$ this agrees with the familiar derivative test ($f' < 0$ stable, $f' > 0$ unstable), but at a point where $f'(x_e) = 0$ only the signs can settle the question.

Logistic baseline. It helps to start with the model without an Allee effect, $f(x) = x(1-x) - q$. For $q < \frac{1}{4}$ it has two steady states,

$$x_\pm = \tfrac{1}{2}\Big(1 \pm \sqrt{1-4q}\Big). \tag{2.2}$$

The upper one is stable. The lower one is unstable, a threshold that exists only because of the catch. As q grows the two move towards each other and meet at $x = \frac{1}{2}$ when $q = \frac{1}{4}$, which gives the well-known maximum constant catch $H_c = rK/4$ [2]. Any larger catch drives every population to zero. With no catch at all, zero is unstable, so even a very small population grows back.

2.3 Model A: the growth curve and the maximum catch

With the Allee factor included, the growth curve G is negative between 0 and a and positive between a and 1. In other words, it dips below zero when the population is small and then rises into a single hill. The top of the hill is where $G'(x) = 0$, which works out to $3x^2 - 2(1+a)x + a = 0$. Taking the larger root,

$$x^* = \frac{1 + a + \sqrt{1 - a + a^2}}{3}, \qquad q_c = G(x^*). \tag{2.3}$$

Steady states are the points where the horizontal line at height q meets this curve, which leaves four cases.

- With no catch ($q = 0$) there are three steady states. Both $x = 0$ and $x = 1$ are stable, while $x = a$ in between is unstable: this is the minimum viable population. Unlike the logistic model, extinction is now a stable outcome.
- For a moderate catch ($0 < q < q_c$) the line cuts the hill twice, at an unstable x_- between a and x^* and a stable x_+ between x^* and 1. A population starting below x_- dies out, while one starting above it settles at x_+.
- At $q = q_c$ the line just touches the top of the hill, and the two states merge into a single semistable point at x^*, called the *fold*.
- For $q > q_c$ the line never meets the hill, and every population collapses.

Once there is any catch at all, zero stops being a steady state, because $f(0) = -q < 0$. A declining population therefore reaches zero in finite time, and we stop the simulation at that point. For $a = 0.2$ we get $x^* \approx 0.706$ and $q_c \approx 0.525$. With $r = 0.5$ per year and $K = 1000$ t, the largest catch the stock can sustain is about 263 t/yr, at a stock of about 706 t.

2.4 Finding the steady states numerically

To compute the steady states for a given catch, we use the shape of G directly:

1. Compute x^* and q_c from Eq. (2.3).
2. If $q > q_c$ there is no positive steady state, and if $q = q_c$ the only one is x^*.
3. If $0 < q < q_c$, f changes sign exactly once between a and x^* and once between x^* and 1. Each root is found with Brent's method, a standard root finder that repeatedly shrinks an interval containing a sign change.
4. Each root is then labelled stable or unstable from the sign of f on either side.

For $a = 0.2$ and $q = 0.3$ this gives $x_- = 0.4431$ (unstable) and $x_+ = 0.9063$ (stable). Very close to q_c the two roots become nearly equal, and if floating-point arithmetic cannot tell them apart, the code reports an error instead of quietly merging them.

2.5 Growth curves and the branch diagram

Figure 2.1 puts this picture on paper. In the logistic panels a threshold only appears once fishing starts. Model A already has one at $x = a$ with no catch, and fishing pushes it higher. Once the catch exceeds q_c, the line passes above the whole hill.

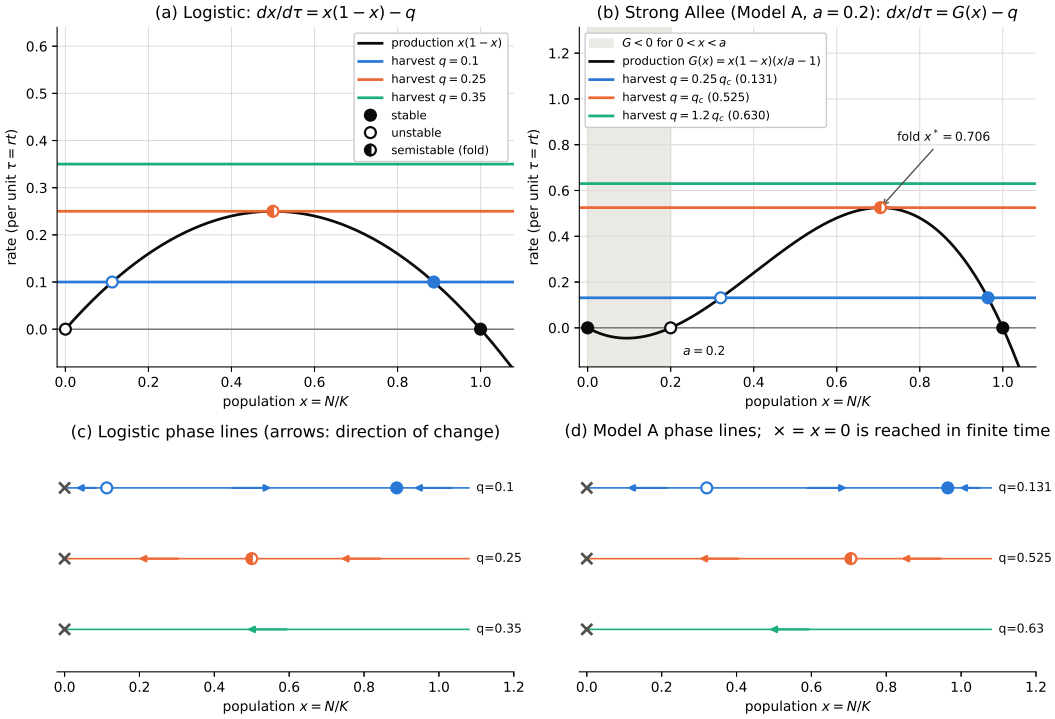


Figure 2.1: Growth curves and phase lines. (a) Logistic growth $x(1-x)$ with catch lines. (b) Strong-Allee growth $G(x)$ for $a = 0.2$, with catches of $0.25q_c$, q_c and $1.2q_c$. Filled, open and half-filled circles mark stable, unstable and semistable states. (c,d) Phase lines: arrows show whether the population grows or shrinks, and × marks $x = 0$, which is reached in finite time when $q > 0$.

Plotting the steady states against the catch gives the branch diagram in Figure 2.2. As the catch increases, the stable branch and the collapse threshold approach each other and

meet at the fold (q_c, x^*). The distance $x_+ - x_-$ between them is the stock's safety margin, and near the fold it shrinks like $\sqrt{q_c - q}$.

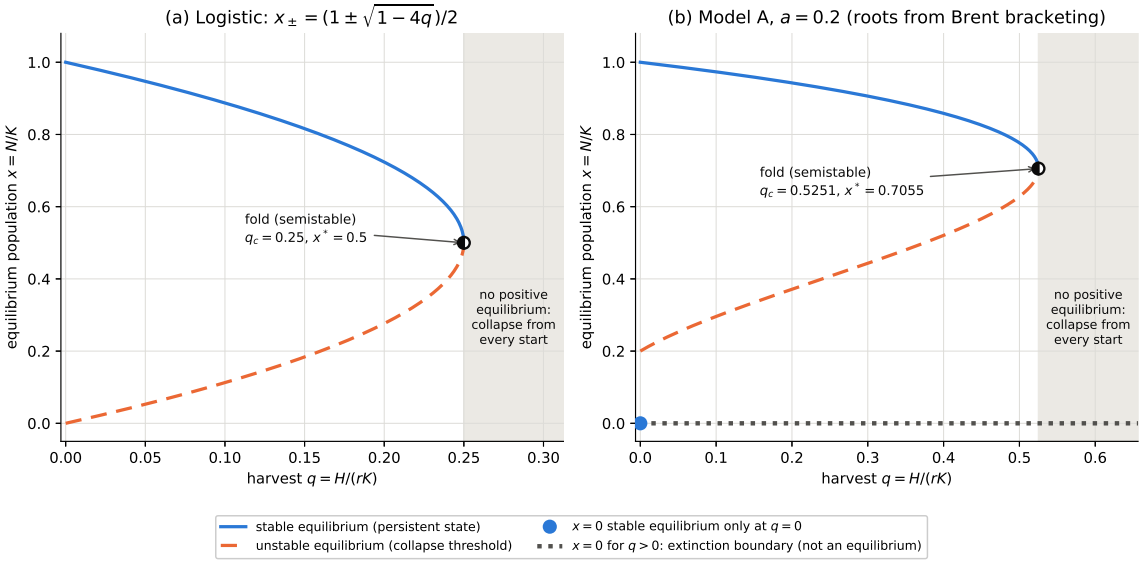


Figure 2.2: Steady states against the scaled catch $q = H/(rK)$. (a) Logistic. (b) Model A with $a = 0.2$. Solid lines are stable, dashed lines unstable, and the half-filled circle is the semistable fold. The shaded region $q > q_c$ has no positive steady state. In (b) the dotted line at $x = 0$ marks extinction, which is not a steady state when $q > 0$.

2.6 Checking stability by simulation

As a check on these labels, we took $q = 0.3$, started the population $\delta = 0.01$ above and below each steady state, and solved the equation with a standard Runge–Kutta solver (RK45 [14]) that stops at $x = 0$. Both starts near x_+ return to it. The start just above x_- climbs back up to x_+, while the one just below falls and reaches zero at $\tau \approx 2.9$ (Figure 2.3). This is exactly what the sign analysis predicted.

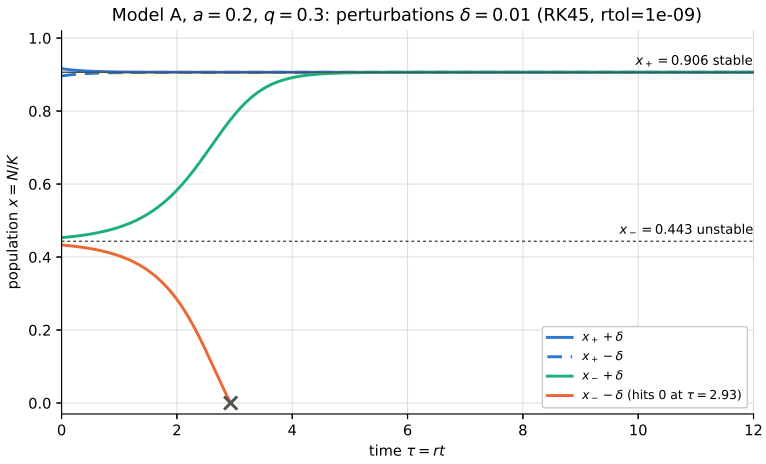


Figure 2.3: Simulation check for Model A with $a = 0.2$ and $q = 0.3$, starting $\delta = 0.01$ above and below each steady state.

2.7 Result and limitations

Main result. As the fixed catch rises, the safe steady state and the collapse threshold draw closer and finally merge at the maximum catch q_c. A disturbance only causes collapse if it pushes the population below the threshold, that is, if it is larger than the gap between the two. Since this gap closes at q_c, fishing near the maximum leaves almost no room for a bad year.

Limitations. The model ignores randomness, which matters most near the threshold. The cubic growth law is a simple phenomenological choice, the parameters are held constant, and the full catch is assumed to be taken whenever $N > 0$. All numbers are illustrative rather than measured from a real stock.

3 Does a smaller catch prevent collapse?

Model A takes a fixed amount each year. Model B takes a fraction of the population: a harvesting rate of 10% per year removes 10 tonnes per year from a 100-tonne stock, but only 2 tonnes per year from a 20-tonne stock. We ask which populations can remain steady, how to compare the harvesting rules fairly, and whether reducing harvesting after collapse allows recovery. All examples here are synthetic, not observations of a real fish stock.

3.1 Steady populations under Model B

As in Section 2, $x = N/K$ is population relative to carrying capacity, $a = A/K$ is the Allee threshold, $\tau = rt$ is scaled time, and $e = h/r$ is harvesting effort. Model B is

$$\frac{dx}{d\tau} = x\left[(1-x)(x/a-1) - e\right]. \tag{3.1}$$

A steady population makes the right-hand side zero. One solution is $x = 0$. For the other solutions, divide by x and rearrange:

$$x^2 - (1+a)x + a(1+e) = 0, \qquad x_\pm = \frac{1+a \pm \sqrt{(1-a)^2 - 4ae}}{2}. \tag{3.2}$$

The roots meet at the critical effort

$$e_c = \frac{(1-a)^2}{4a}, \qquad h_c = re_c. \tag{3.3}$$

- **Below e_c:** the lower root x_- is the recovery threshold and the upper root x_+ is stable. Starting below x_- leads towards zero; starting above it leads towards x_+.
- **At e_c:** the roots meet. The steady state attracts from above but not from below; this is called *semistable*.
- **Above e_c:** no positive steady population remains.

To see why, factor the equation as $f_B = -x(x-x_-)(x-x_+)/a$. Its signs on the three positive-population intervals are negative, positive and negative. Also $f_B'(0) = -(1+e) < 0$, so zero stays stable even when harvesting stops.

For example, with $a = 0.2$ and $e = 0.4$, the recovery threshold is about $0.317K$ and the stable population is about $0.883K$. Here $e_c = 0.8$. Figure 3.1 shows how these steady populations change with effort.

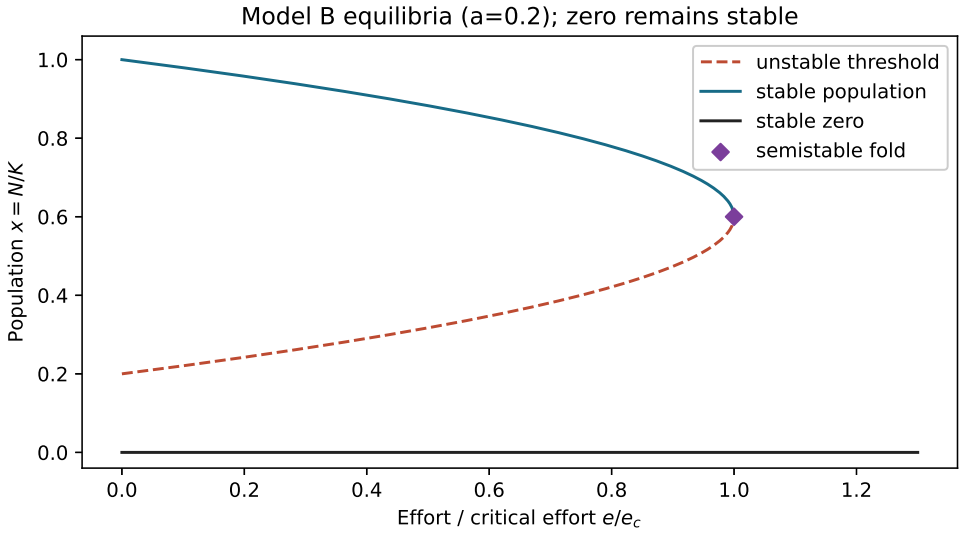


Figure 3.1: Model B steady populations for $a = 0.2$. Solid lines are stable; the dashed line is the recovery threshold. The diamond marks where the positive roots meet. Zero remains stable.

3.2 Compare harvesting fairly

A fixed catch H (tonnes/year) and an effort h (per year) have different units. We therefore use two comparisons:

1. **Same starting catch:** choose $H = hN_0$. Both models remove the same amount initially; Model B's catch then changes with population. For the example, set $H = H_c^A/2$ and give the logistic model this same catch.

2. **Same fraction of the limit:** use half of each model's own critical harvesting level. This compares equal fractions of the harvesting limits, but the starting catches differ.

Use $r = 0.5$ per year, $K = 1000$ tonnes, $A = 200$ tonnes and a starting population of 900 tonnes. Follow the population for 60 years. The logistic model from Section 2 is included as a baseline.

Mean catch is total catch divided by 60 years. Total catch is the area under the catch-rate curve, $\int_0^{60} hN(t)\,dt$ for B. For a fixed catch, harvesting stops at extinction. The mean includes the years after shutdown. Table 3.1 gives the compact comparison.

Table 3.1: Synthetic comparison from 900 tonnes over 60 years. Catch rates are in tonnes/year; outcomes are the predicted long-term behaviour.

Comparison	Model	Starting catch	Mean catch	Outcome
Same starting catch	A	131.28	131.28	Persists
Same starting catch	B	131.28	134.00	Persists
Same starting catch	Logistic	131.28	103.35	Collapses
Half of each limit	A	131.28	131.28	Persists
Half of each limit	B	180.00	176.61	Persists
Half of each limit	Logistic	62.50	62.50	Persists

At the same starting catch, A and B persist but the logistic stock collapses: 131.28 tonnes/year exceeds its maximum fixed catch of 125 tonnes/year. At half of each model's limit, all three persist. This does *not* prove that Allee growth is safer. The growth curves differ, and the conclusion depends on how the comparison is set up. The full experiment also includes lower starting populations and harvesting above the critical level.

3.3 Does the population recover when harvesting falls?

Start at $N = K$, increase harvesting in 12 steps to 1.3 times its critical level, hold it there, then decrease it to zero. Carry the population forward at every step: never restart it at the turning point. Repeat with two dwell times, 1 and 3 scaled time units per ordinary step. After harvesting returns to zero, compare:

- **No intervention:** let the population evolve on its own.
- **Restocking:** add $0.6K = 600$ tonnes, shown separately by the dashed population lines in Figure 3.2.

The peak hold lasts 12 scaled time units. After returning to zero harvest, wait 15 units before the addition, then follow both runs for another 20 units. One scaled time unit equals two years in this example.

For A, reaching zero triggers harvest shutdown. For B, a positive population approaches zero without reaching it in finite time; $x \leq 10^{-6}$ is only a smallness marker, not an extinction event.

Main conclusion. Reducing harvesting does not guarantee recovery. In both tested schedules, the unassisted populations remain near zero. Below the Allee threshold, natural growth is negative even without harvesting. Adding 600 tonnes crosses the threshold and allows recovery towards K.

This is **history dependence with a recovery barrier**, not a closed loop that recovers by itself. The slower schedule also means longer exposure to harvesting, so it does not isolate the effect of speed alone.

Checks and limits. Using tighter solver settings gives almost identical results. Very small populations and starting points close to the recovery threshold need care because numerical precision is limited. These are simplified examples, not predictions for a real fish stock, and they do not show that one harvesting model is always safer than the other.

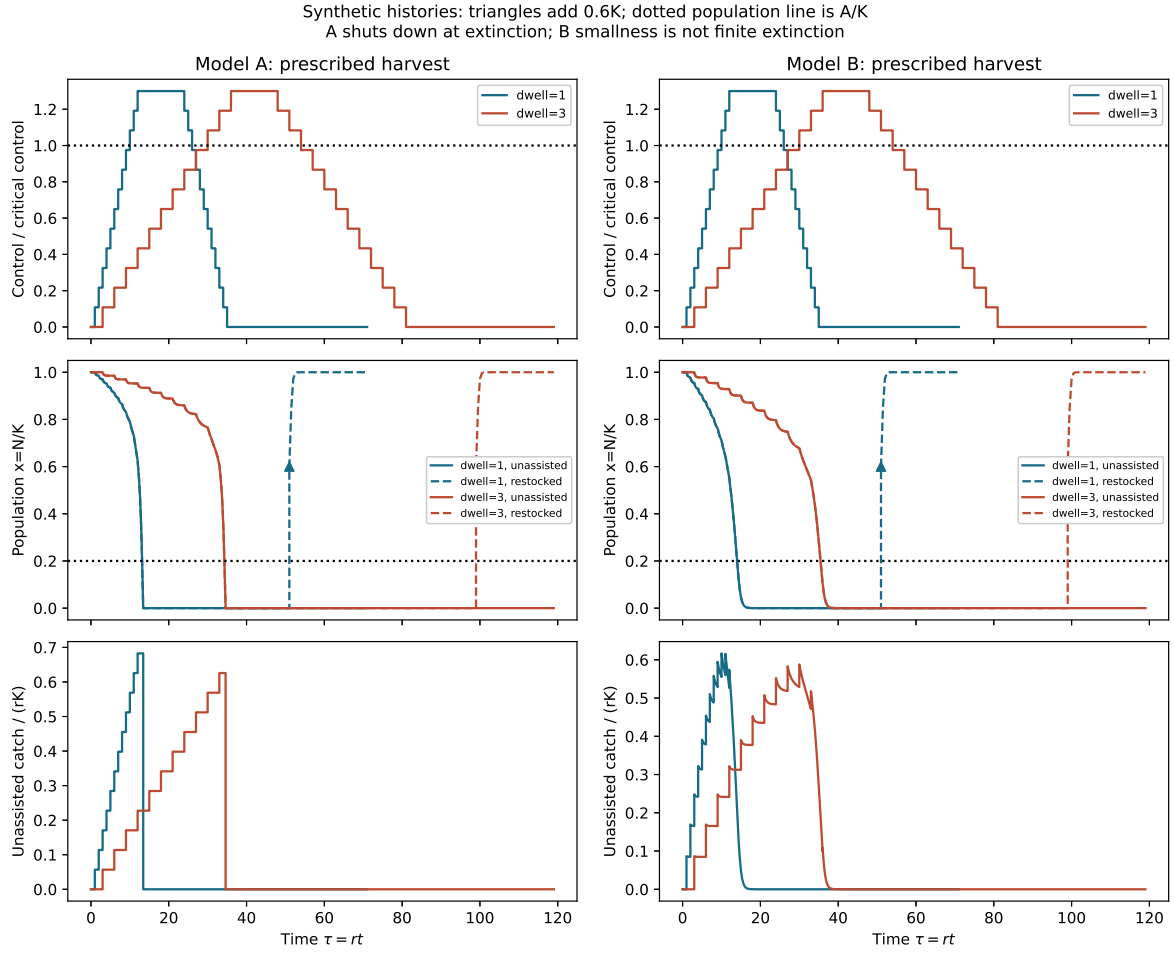


Figure 3.2: Two synthetic harvesting schedules. Top: prescribed harvesting relative to its critical level. Middle: population without intervention (solid) and with restocking (dashed; triangles mark the addition). The dotted population line is $a = 0.2$, the recovery threshold at zero harvest. Bottom: actual catch without intervention, including A's shutdown. Time is $\tau = rt$.

4 Critical Harvesting Rates and Extinction Analysis

In Sections 2 and 3, we studied the equilibrium points and stability of Models A and B. We now calculate the critical harvesting rates, examine how the initial population affects survival, and determine the time to extinction when a population collapses. We also investigate slow decline near the critical harvesting rate.

We use the dimensionless variables introduced earlier:

$$x = \frac{N}{K}, \qquad \tau = rt, \qquad a = \frac{A}{K}, \qquad q = \frac{H}{rK}, \qquad e = \frac{h}{r}.$$

The two harvesting models are

$$\text{Model A:} \qquad x' = G(x) - q, \tag{4.1}$$

$$\text{Model B:} \qquad x' = G(x) - ex, \tag{4.2}$$

where

$$G(x) = x(1-x)\left(\frac{x}{a} - 1\right), \qquad x' = \frac{dx}{d\tau}.$$

4.1 Critical Harvesting Rates

Model A: Constant harvesting. At a positive equilibrium, natural growth balances the fixed catch: $G(x) = q$. The largest constant catch that can be balanced is therefore the maximum of $G(x)$ for $x \geq 0$. Expanding and differentiating gives

$$G(x) = -\frac{x^3}{a} + \frac{(1+a)x^2}{a} - x, \qquad G'(x) = \frac{-3x^2 + 2(1+a)x - a}{a}.$$

Setting $G'(x) = 0$ gives

$$x = \frac{1 + a \pm \sqrt{1 - a + a^2}}{3}.$$

The larger stationary point lies between a and 1. Writing it as x_*, we have

$$x_* = \frac{1 + a + \sqrt{1 - a + a^2}}{3}, \qquad G''(x_*) = -\frac{2\sqrt{1 - a + a^2}}{a} < 0.$$

Thus the critical constant catch is

$$q_c = G(x_*), \qquad H_c = rKq_c. \tag{4.3}$$

The corresponding population in the original units is

$$N_* = Kx_* = \frac{K + A + \sqrt{K^2 - KA + A^2}}{3}.$$

Model B: Proportional harvesting. At a positive equilibrium, $G(x) = ex$. Dividing by $x > 0$ gives

$$e = \frac{G(x)}{x} = (1-x)\left(\frac{x}{a} - 1\right) = P(x).$$

Here we maximise growth per unit population, rather than total growth. Since

$$P'(x) = \frac{1 + a - 2x}{a}, \qquad P''(x) = -\frac{2}{a} < 0,$$

the maximum occurs at $x = (1 + a)/2$. Hence

$$e_c = \frac{(1-a)^2}{4a}, \qquad h_c = \frac{r(K-A)^2}{4AK}. \tag{4.4}$$

Above their respective critical harvesting rates, neither model has a positive equilibrium. Below these rates, positive equilibria exist, but the initial population still determines whether the population survives.

For the numerical examples in this section, we use $r = 0.7$, $K = 100$ and $A = 20$. We obtain

$$N_* \approx 70.55050, \qquad H_c \approx 36.75966, \qquad h_c = 0.56.$$

For comparison, the standard logistic model with constant harvesting has $H_{c,\text{logistic}} = rK/4 = 17.5$. However, the logistic and Allee models have different growth functions. The larger critical harvesting rate does not mean that populations with a strong Allee effect are always more resistant to collapse. Also, H and h have different units, so their numerical values cannot be compared directly.

We checked the analytical thresholds by independently maximising the corresponding growth functions numerically.

4.2 Initial Population and Basins of Attraction

Even below the critical harvesting rate, a population may collapse if its initial size is too small. A *basin of attraction* is the set of initial populations that approach the same long-term state.

For Model A with $0 < q < q_c$, there are two positive equilibria, $x_- < x_+$. The sign of $G(x) - q$ is negative below x_-, positive between the roots, and negative above x_+. Therefore x_- is unstable and x_+ is stable. A population starting below x_- reaches zero, whereas a population starting above it approaches x_+.

For Model B, the positive equilibria are

$$x_\pm = \frac{1 + a \pm \sqrt{(1-a)^2 - 4ae}}{2}, \qquad 0 \le e < e_c.$$

The lower equilibrium is unstable and the upper one is stable. Model B also has a stable zero equilibrium: if $f_B(x) = G(x) - ex$, then

$$f_B'(0) = -(1 + e) < 0.$$

At the critical harvesting rate, the two positive equilibria merge at a *fold*. For Model A this occurs at x_*, and for Model B at $(1 + a)/2$. The merged equilibrium attracts populations from above but not from below. This one-sided behaviour is called *semistability*.

Table 4.1 summarises the outcomes.

With zero harvesting, $x_0 = a$ is also an equilibrium. For Model A with $q > 0$, however, $G(0) - q = -q < 0$: zero is a boundary at which we stop the physical solution, not an equilibrium of the original ODE. A population on a collapsing trajectory reaches this boundary in finite time. Under Model B, a declining positive population approaches zero asymptotically instead.

Figure 4.1 shows the basin diagrams. The blue regions show initial conditions leading to a positive equilibrium. The orange region in Model A shows finite-time extinction, while the yellow region in Model B shows asymptotic decline. The lower equilibrium separates collapse from persistence below the critical harvesting rate. We classify the basins from the signs of

Table 4.1: Long-term outcomes for positive initial populations. For below-critical harvesting, the harvest is positive. A population starting exactly at an equilibrium remains there.

Harvesting	**Model A**	**Model B**
Zero	For $x_0 < a$, approaches zero; for $x_0 > a$, approaches 1.	Same behaviour as Model A.
Below critical	For $x_0 < x_-$, reaches zero; for $x_0 > x_-$, approaches x_+.	For $x_0 < x_-$, approaches zero; for $x_0 > x_-$, approaches x_+.
At critical	Below the merged root, reaches zero; above it, approaches the merged root.	Below the merged root, approaches zero; above it, approaches the merged root.
Above critical	Every finite $x_0 > 0$ reaches zero.	Every $x_0 > 0$ approaches zero.

the equations rather than simply using the value at the end of a finite simulation. Conditions extremely close to an unstable or merged equilibrium are marked unresolved when rounding prevents a reliable classification.

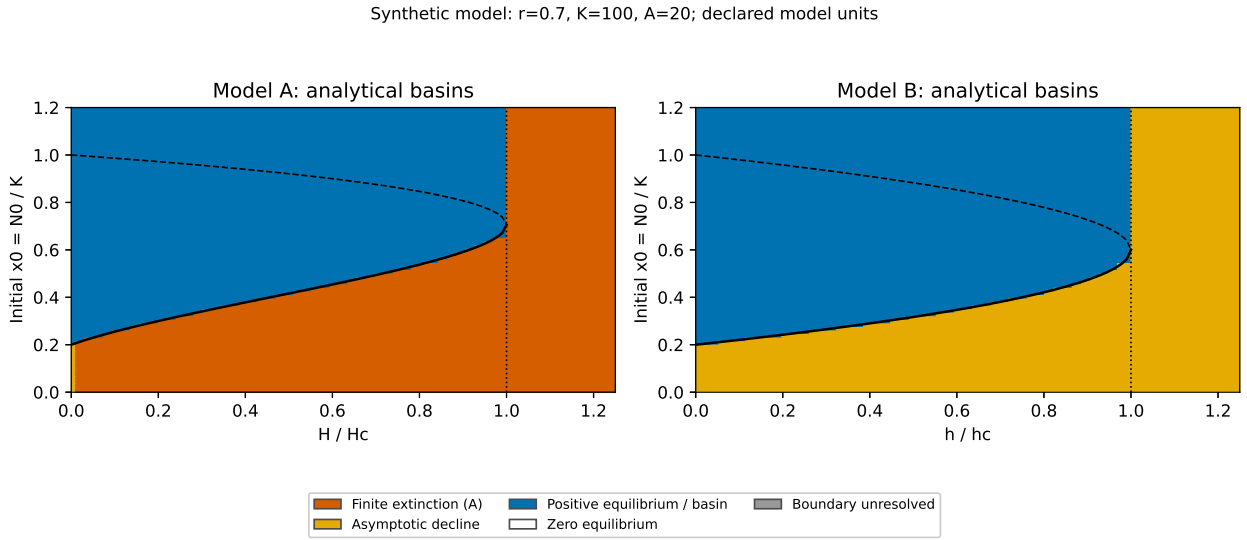


Figure 4.1: Basins of attraction for Models A and B with $a = 0.2$. The horizontal axis in each panel is the harvest divided by that model's critical rate; the vertical axis is the initial population $x_0 = N_0/K$. Solid curves mark lower equilibria, dashed curves mark upper equilibria, and vertical dotted lines mark the critical harvesting rates.

4.3 Time to Extinction

For a Model A trajectory that reaches zero, the extinction time follows from separation of variables:

$$\frac{dx}{d\tau} = G(x) - q, \qquad \tau_{\text{ext}} = \int_{x_0}^{0} \frac{dx}{G(x) - q} = \int_{0}^{x_0} \frac{du}{q - G(u)}.$$

To express this in the original units, recall that

$$x = \frac{N}{K}, \qquad \tau = rt, \qquad q - G(x) = \frac{H - g(N)}{rK},$$

where

$$g(N) = rN\left(1 - \frac{N}{K}\right)\left(\frac{N}{A} - 1\right).$$

Since $dx = dN/K$, the substitution gives

$$\tau_{\text{ext}} = r\int_0^{N_0} \frac{dN}{H - g(N)}.$$

Thus the dimensional extinction time is

$$t_{\text{ext}} = \frac{\tau_{\text{ext}}}{r} = \int_0^{N_0} \frac{dN}{H - g(N)}. \tag{4.5}$$

This integral gives a finite extinction time only when the trajectory reaches zero without encountering an equilibrium. We therefore classify the initial condition before evaluating it. If an equilibrium lies along the path to zero, the denominator vanishes and the integral cannot be used to assign a finite extinction time.

For $q > q_c$, we have $G(x) \le q_c$ for all $x \ge 0$. Consequently,

$$q - G(x) \ge q - q_c > 0, \qquad \tau_{\text{ext}} \le \frac{x_0}{q - q_c}.$$

Every finite positive initial population therefore reaches zero in finite time for Model A above its critical constant catch. At $q = q_c$, a start below x_* still reaches zero; a start above x_* approaches the merged equilibrium instead.

For Model B, a positive trajectory cannot reach its zero equilibrium in finite time because the ODE is smooth and solutions are unique. The same holds for Model A when $q = 0$. A numerical time to a small positive cutoff is not an exact extinction time in either of these situations.

We evaluate the extinction-time integral using numerical quadrature [13]. Close to a basin boundary or the fold, its denominator can become small and the calculation requires care. Our implementation uses changes of variable near difficult points. These numerical estimates are checked independently below.

4.4 Numerical Method and Accuracy Checks

We solve the population ODEs using SciPy's RK45 method, based on the adaptive Dormand–Prince Runge–Kutta pair of orders five and four [5, 14]. The difference between the two estimates is used to adjust the step size. Our reference settings are relative tolerance 10^{-9}, absolute tolerance 10^{-11} and maximum dimensionless step size 0.1.

For Model A under positive catch, the solver stops at the first downward crossing of zero. We do not continue the original ODE into negative populations.

For six Model A extinction cases, we compare the separated integral with RK45 at two tolerances and with the independent DOP853 method. The cases cover harvesting below, at and above the critical rate, with initial populations chosen on extinction trajectories. We use the acceptance condition

$$|\Delta\tau| \le 2 \times 10^{-7} + 5 \times 10^{-7}\tau_{\text{ext}}.$$

Table 4.2 lists each case. Here $x_0 = N_0/K$, and q/q_c is the fraction of the critical constant catch.

All six cases passed, with the largest recorded difference approximately 4.56×10^{-8} in dimensionless time. This supports numerical agreement for the tested cases, rather than a global accuracy guarantee.

Table 4.2: Six Model A extinction-time checks. The largest difference is the maximum absolute difference between the quadrature result and the tested ODE integrations, in dimensionless time.

q/q_c	x_0	τ_{ext}	$\max \lvert\Delta\tau\rvert$	Tolerance
0.25	0.10000	0.6198	2.3×10^{-10}	5.1×10^{-7}
0.75	0.35769	0.9957	1.6×10^{-9}	7.0×10^{-7}
1.00	0.56440	1.9303	6.4×10^{-9}	1.2×10^{-6}
1.01	1.10000	19.9106	4.6×10^{-8}	1.0×10^{-5}
1.05	1.10000	8.6537	3.8×10^{-8}	4.5×10^{-6}
1.20	1.10000	4.0293	3.9×10^{-8}	2.2×10^{-6}

Figure 4.2 shows Model A extinction times for different initial populations and constant catches. In the left panel, the time grows as the starting population approaches the unstable equilibrium from below. Above that boundary, the population persists and there is no finite extinction time to plot.

The right panel fixes the initial population and varies the constant catch. For a population that reaches zero, a larger catch produces a shorter extinction time. When the initial population lies above the fold population, extinction occurs only for $H > H_c$, and the time can be very large just above this threshold. Both vertical axes are logarithmic. The underlying analysis contains 427 records, including cases without a reported finite extinction time.

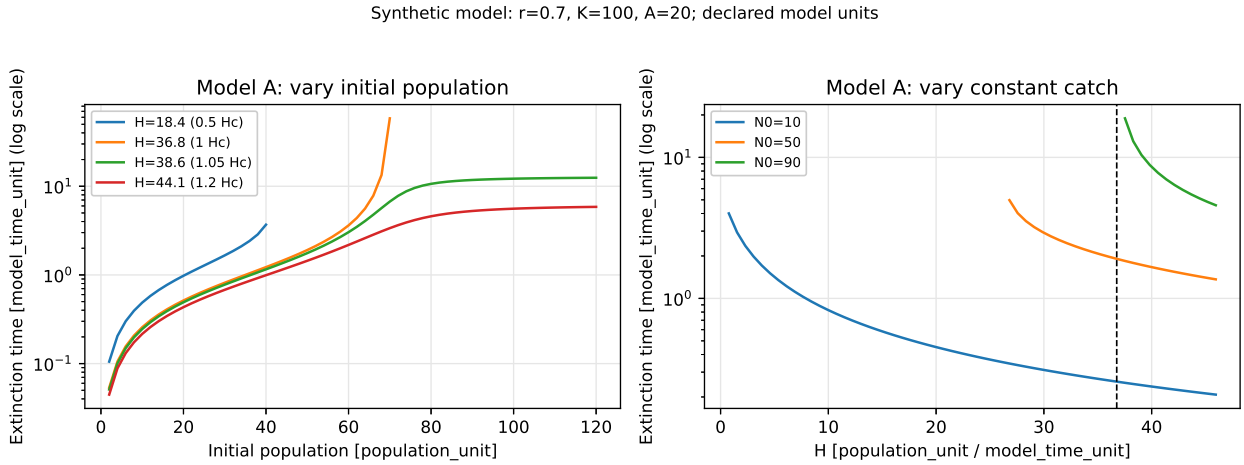


Figure 4.2: Model A extinction time against initial population (left) and constant harvesting rate (right), with $r = 0.7$, $K = 100$ and $A = 20$. Both vertical axes are logarithmic. Gaps indicate cases without a reported finite extinction time; the dashed line on the right marks H_c.

4.5 Slow Decline Near the Critical Harvesting Rate

Extinction need not happen quickly when q is only slightly larger than q_c. Although no positive equilibrium remains, natural growth almost balances the constant catch near the former equilibrium population x_*. The net decline is small there, so the population can spend a long time in this region before falling to zero. This slow passage is called a *bottleneck*.

Figure 4.3 shows three trajectories starting at $x_0 = 1.1$, with $a = 0.2$ and harvesting rates just above q_c. The closer the catch is to its critical value, the longer the delay. Each

trajectory ends at its calculated zero event. We report this numerical behaviour without fitting a separate asymptotic scaling law.

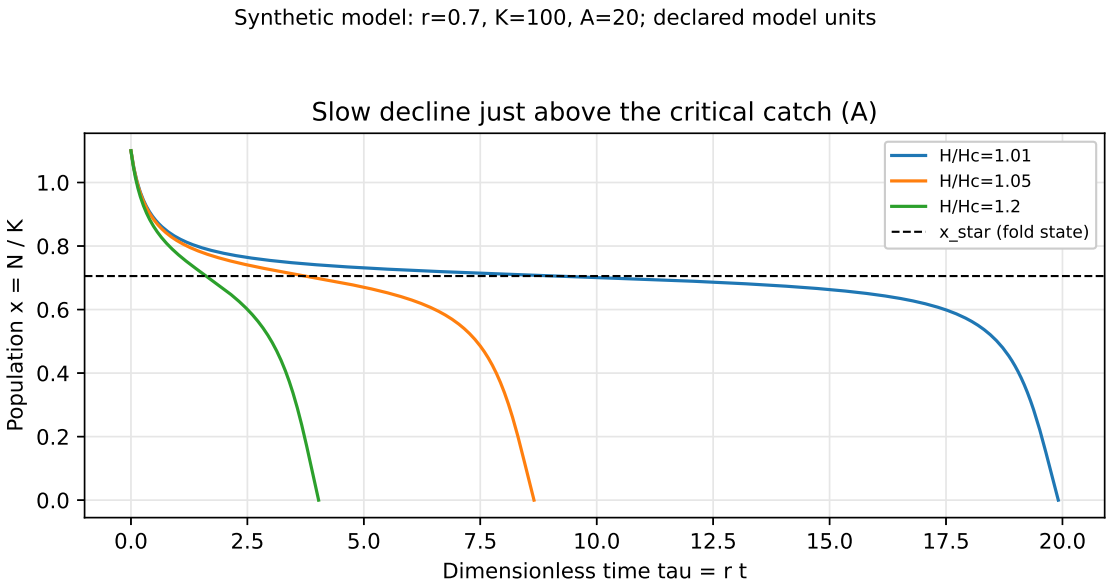


Figure 4.3: Model A trajectories slightly above the critical constant catch, with $a = 0.2$ and $x_0 = 1.1$. Time and population are dimensionless. The dashed line shows the population at the former fold. Each curve ends when the population reaches zero.

4.6 Synthetic Reference Data and Limitations

We also generated six clean synthetic reference trajectories using RK45. Each contains 101 sampled population values, giving 606 values in total. For each model, the three cases are regular persistence, behaviour near the critical harvesting rate, and decline or extinction. The Model A declining case reaches zero in finite time, whereas the Model B declining case approaches zero asymptotically.

The generating parameters, initial populations and solver settings are recorded in the project configuration files. The six cases are named `a_regular`, `a_near_fold`, `a_extinction`, `b_regular`, `b_near_fold` and `b_decline`. All six passed comparisons with DOP853 and tighter-tolerance RK45 integrations.

Figure 4.4 shows these reference trajectories. The complete data-generation scripts, configurations and numerical results are available in our GitHub project repository.

These trajectories form an early numerical reference dataset. The forward PINN experiments use separately generated numerical reference solutions for evaluation, while the inverse experiments use their own observation, noise and data-splitting procedures. Any zero values stored after a Model A extinction event represent an absorbing extinction convention, not a continuation of the original ODE with constant catch.

Main result. Both the harvesting rate and initial population matter for survival. Positive constant harvesting can produce finite-time extinction, whereas declining positive populations under proportional harvesting approach zero asymptotically. Collapse can also be delayed near the critical constant harvesting rate.

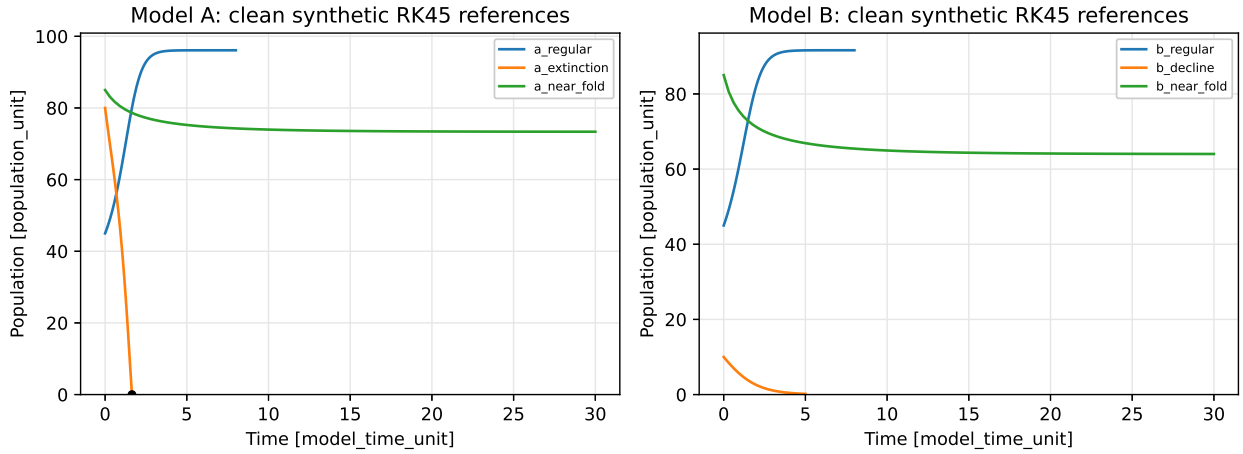


Figure 4.4: Six synthetic RK45 reference trajectories for Models A and B, showing regular persistence, near-critical behaviour and decline or extinction.

Limitations. The calculations assume constant parameters and a well-mixed population, without environmental randomness, age structure or time delays. All trajectories are synthetic. The critical rates and extinction times are consequences of the specified models, not predictions for a particular real-world population.

5 Forward PINNs: solving the equation with a neural network

Question. If a small neural network is given only the differential equation and the starting population, how well can it predict the population over time, compared with a standard numerical solver? All data in this section are synthetic and scaled ($x = N/K$, $\tau = rt$). The time interval is $\tau \in [0, T]$ with $T = 10$ (for a Model A run that goes extinct, T is the extinction time) and $x \in [0, 1]$. The setting described here is the *same* one used later for the inverse problem (Section 6).

5.1 The idea

A physics-informed neural network (PINN) [11] is a neural network $\hat{x}(\tau)$ that is trained to satisfy a differential equation rather than to match measured data. We write both models as $\dot{x} = f(x)$, with $f(x) = G(x) - q$ for Model A and $f(x) = G(x) - ex$ for Model B. For any candidate $\hat{x}$, the *residual*

$$R(\tau) = \frac{d\hat{x}}{d\tau} - f(\hat{x}) \tag{5.1}$$

measures how badly the equation is violated at time τ. If $R \equiv 0$ and the starting value is right, $\hat{x}$ is the solution. Training therefore means making R small at many chosen times, called *collocation points*. The derivative $d\hat{x}/d\tau$ is obtained exactly by automatic differentiation, so no finite-difference step is involved. No solver solution is ever used as a training label.

5.2 Building the starting condition into the network

A fully connected network $N(s;\theta)$ with layer sizes $(1, 32, 32, 1)$ and the tanh activation is evaluated at the rescaled time $s = \tau/T \in [0, 1]$. We define

$$\hat{x}(\tau) = x_0 + s\,N(s;\theta), \tag{5.2}$$

so that $\hat{x}(0) = x_0$ *exactly, for any weights* [9]. No separate penalty for the starting value is needed, and the optimiser can spend all its effort on the equation. By the chain rule and the product rule, $d\hat{x}/d\tau = T^{-1}\big(N + s\,\partial_s N\big)$. The code uses the JAX library [1] in 64-bit arithmetic.

5.3 Loss and training

The loss is the mean squared residual over $n = 300$ fixed, uniformly spaced collocation times $\tau_i \in [0, T]$:

$$\mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n} R(\tau_i)^2. \tag{5.3}$$

For Model A with extinction, the training interval ends at the extinction time, so the zero tail never enters the residual. Training has two phases that are common practice for PINNs. First the Adam optimiser [8] runs for 1000 iterations at learning rate 10^{-3} to reach a good region of weight space (about 2/7 of the 3500-iteration budget). Then L-BFGS [10], a method that uses curvature information, runs for 2500 iterations with its tolerance-based stop switched off, so that it always uses the full budget and is not ended early by a loose stopping rule.

5.4 How the answer is scored

The reference is a dense DOP853 solution (relative tolerance 10^{-12}) [6], kept separate from training. We report the root-mean-square error (RMSE), the mean absolute error, the *relative* L^2 *error* (the size of the error relative to the size of the solution), the largest error, and the largest error where $x_{\mathrm{ref}} < 0.05$ (the near-zero error, since a small absolute error can still be a large relative one there). A prediction containing NaN or infinity is never scored and is labelled invalid, and a run that raises an exception is recorded as failed. A run is *inaccurate* if its relative L^2 error exceeds 5×10^{-2}; this threshold was fixed beforehand.

5.5 Choosing the setting

The network setting was chosen by trial and error. We trained the forward PINN with three activation functions (tanh, swish [12] and sigmoid) at different learning rates, using the same fixed seeds (0, 1 and 42) and the same six test cases for every candidate. The combination of activation function and learning rate that gave the most accurate solutions was adopted: tanh with a network learning rate of 10^{-3}, together with the other choices described above (a $(1, 32, 32, 1)$ network, 300 collocation points and the Adam then L-BFGS schedule). This single setting is used for all the results below and, unchanged, for the inverse problem in Section 6.

Because the setting was selected on seeds 0, 1 and 42, accuracy on those seeds is optimistic. To check that the network really learns the model, and not just those particular runs, we repeated the whole experiment with the frozen setting on three further seeds (11, 22 and 33) that played no part in the selection.

5.6 Results

Table 5.1 gives the final run with the frozen setting (tanh, $(1, 32, 32, 1)$, learning rate 10^{-3}, 300 points, seeds 0, 1, 42). All 18 runs are accurate. The relative L^2 error ranges from 2.9×10^{-7} to 4.1×10^{-3}, with a median of 6.4×10^{-6}. The hardest case is Model A near the fold, where the population lingers near the old steady state before collapsing (Section 4). On the three fresh seeds (11, 22, 33), which played no part in the selection, all 18 runs were again accurate (errors 1.4×10^{-7} to 2.3×10^{-3}, median 5.8×10^{-6}), so 36 of 36 runs are accurate in all.

Table 5.1: Forward PINN, final setting: relative L^2 error against the DOP853 reference for three seeds per case (seeds 0, 1, 42).

Case	Smallest	Median	Largest
Model A regular	1.8×10^{-6}	1.4×10^{-5}	2.0×10^{-5}
Model A near-fold	1.7×10^{-5}	2.1×10^{-3}	4.1×10^{-3}
Model A extinction	2.9×10^{-7}	7.4×10^{-7}	7.4×10^{-7}
Model B regular	1.0×10^{-6}	5.2×10^{-6}	7.7×10^{-6}
Model B near-fold	5.4×10^{-7}	8.5×10^{-7}	5.2×10^{-6}
Model B decay	1.6×10^{-5}	2.2×10^{-5}	6.6×10^{-5}

Figure 5.1 compares the PINN with the reference, and Figure 5.2 shows the error and the loss during training; the vertical line marks the hand-over from Adam to L-BFGS.

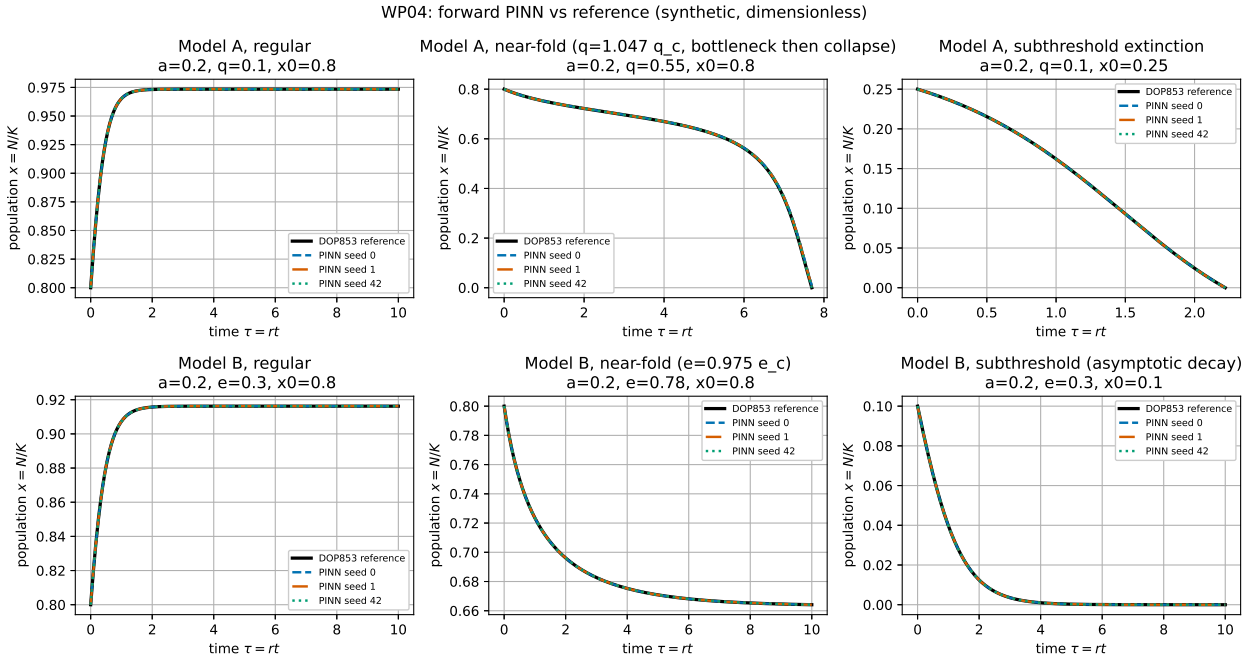


Figure 5.1: Forward PINN prediction against the DOP853 reference for the six cases (final setting). All quantities are scaled.

Some further observations:

- **Negative values.** Five of the 18 runs predict slightly negative populations near zero: Model B decay for seeds 1 and 42 (23 and 61 of 1001 sample points) and one sample each for Model A extinction (two runs) and Model A near-fold (one run). The near-zero

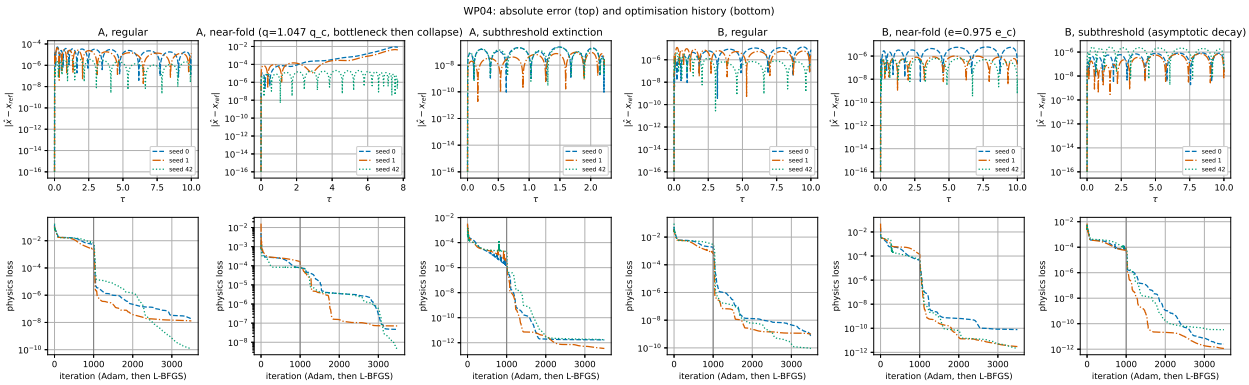


Figure 5.2: Pointwise error and training loss for the six cases. The vertical line in the bottom row marks the hand-over from Adam to L-BFGS.

error (the largest absolute error where $x_{\text{ref}} < 0.05$) is 8.9×10^{-8} to 1.9×10^{-7} for Model A extinction, 7.3×10^{-7} to 2.6×10^{-6} for Model B decay and 4.8×10^{-6} to 8.6×10^{-3} for Model A near-fold.

- **The iteration budget is binding.** In every run L-BFGS stopped at its 2500-iteration limit, not through an abnormal exit; longer runs were not tried.
- **Speed.** The classical RK45 solver matches the reference to between 6×10^{-11} and 1.4×10^{-9} in 4 to 10 milliseconds. Each PINN run took 3.2 to 4.6 seconds. For a single scalar equation the classical solver is clearly the better tool; the value of the PINN lies in being the engine for the inverse problem (Section 6).
- **Extinction time is not predicted.** For Model A the horizon T is the extinction time taken from the reference solver. The forward PINN does *not* predict the extinction time itself.

5.7 Limits

There is one parameter set per regime, three seeds per set, and one computer; individual results depend on the platform and library versions, which are recorded in the run manifests. The setting was selected on one set of seeds and confirmed on another. Most importantly, a small training loss is not by itself evidence of accuracy: the early failures had small loss at the collocation points and large error between them. The reference comparison, not the loss, is what establishes accuracy.

6 Inverse PINNs, Identifiability, and Held-Out Validation

This section estimates unknown parameters of the strong-Allee harvesting models from discrete population observations. It addresses two questions: (i) whether the physical parameters are uniquely determined by ideal trajectories (structural identifiability), and (ii) how accurately conventional ODE least squares and an inverse physics-informed neural network (PINN) recover parameters and predict a held-out time window.

6.1 Model B structural nonidentifiability and counterexample

Under Model B, harvesting is proportional to population, at a constant effort level h:

$$\frac{dN}{dt} = rN\left(1 - \frac{N}{K}\right)\left(\frac{N}{A} - 1\right) - hN. \tag{6.1}$$

Expanding the cubic gives

$$\frac{dN}{dt} = -(r+h)N + r\left(\frac{1}{A} + \frac{1}{K}\right)N^2 - \frac{r}{AK}N^3 = c_1 N + c_2 N^2 + c_3 N^3, \tag{6.2}$$

where

$$c_1 = -(r+h), \qquad c_2 = r\left(\frac{1}{A} + \frac{1}{K}\right) = \frac{r(A+K)}{AK}, \qquad c_3 = -\frac{r}{AK}. \tag{6.3}$$

At a single fixed effort, the trajectory therefore depends on the four physical parameters (r, A, K, h) only through the three composite coefficients (c_1, c_2, c_3). The parameters cannot, in general, be uniquely recovered from such a trajectory: Model B is *structurally nonidentifiable* when all four parameters are unknown and effort is held fixed.

We observe an explicit counterexample:

$$(r, A, K, h)_1 = (1.0,\ 0.2,\ 1.0,\ 0.5), \tag{6.4}$$

$$(r, A, K, h)_2 = (1.35,\ 0.3,\ 0.9,\ 0.15). \tag{6.5}$$

Both sets produce

$$(c_1, c_2, c_3) = (-1.5,\ 6.0,\ -5.0), \qquad \dot{N} = -1.5N + 6N^2 - 5N^3. \tag{6.6}$$

They consequently define the same vector field and the same trajectories from every common initial population. The numerical demonstration overlays the two parameter sets for three initial populations, $N(0) = 0.4$, 0.7, and 1.1; the corresponding curves coincide (Figure 6.1).

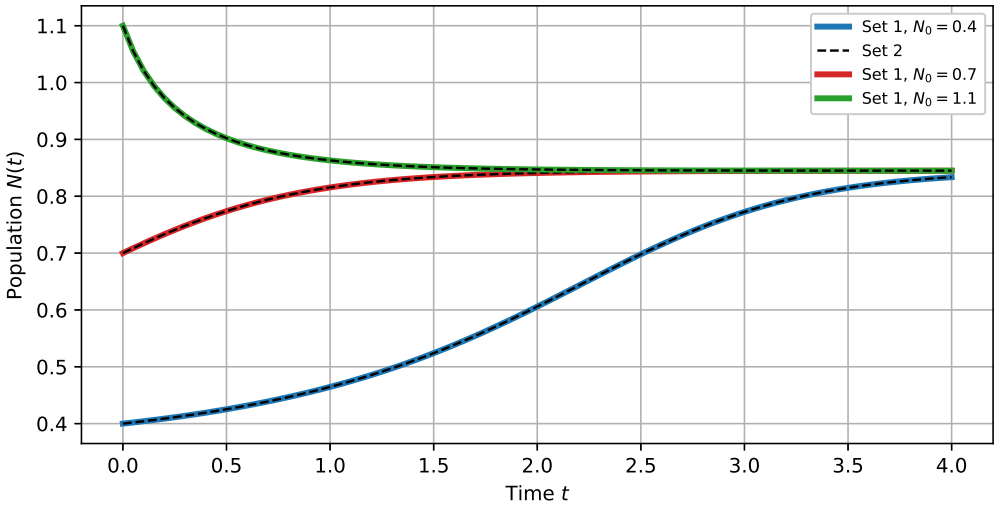


Figure 6.1: Structural nonidentifiability in Model B. Two distinct parameter sets produce the same three polynomial coefficients and therefore overlapping population trajectories from each of the three initial populations.

Using several initial populations at the same effort does not resolve the ambiguity: all such trajectories are solutions of the same vector field and do not provide a fourth independent coefficient. An independently known parameter, such as the carrying capacity K, is needed to constrain the problem. The single-case demonstrations below use Model A and treat K and the initial population as known; the multi-regime benchmark also includes Model B.

6.2 Observations, noise generation, and frozen 60/20/20 splits

For the single-trajectory demonstrations, the reference is Model A with

$$r = 0.7, \qquad K = 100, \qquad A = 20, \qquad H = 10, \qquad N(0) = 45. \tag{6.7}$$

The clean trajectory is sampled at 101 equally spaced observation times on $t \in [0, 8]$. Noisy observations are generated by

$$N_{\text{obs}}(t_i) = N_{\text{clean}}(t_i) + \sigma N_{\text{ref}} \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, 1), \qquad N_{\text{ref}} = 100. \tag{6.8}$$

The external reference scale is fixed at $N_{\text{ref}} = 100$; thus $\sigma = 0.01$ corresponds to a noise standard deviation of one population unit. Observations are not silently clipped, including when additive noise would produce a negative value. The clean and 1% noisy Model A demonstrations use noise seed 11.

The 101 observation times are partitioned into contiguous, non-overlapping time windows:

- **Training:** 61 points, $0 \leq t \leq 4.80$ (60%). These are the only observations used in the data-fit objective.
- **Validation:** 20 points, $4.88 \leq t \leq 6.40$ (20%). These observations are excluded from the training gradients. The clean demonstration and the frozen benchmark disable validation-based checkpoint selection; the separate noisy demonstration enables it.
- **Held-out test:** 20 points, $6.48 \leq t \leq 8.00$ (20%). These observations remain untouched until the post-training evaluation.

The split counts and endpoints are shown in the figure 6.2 below.

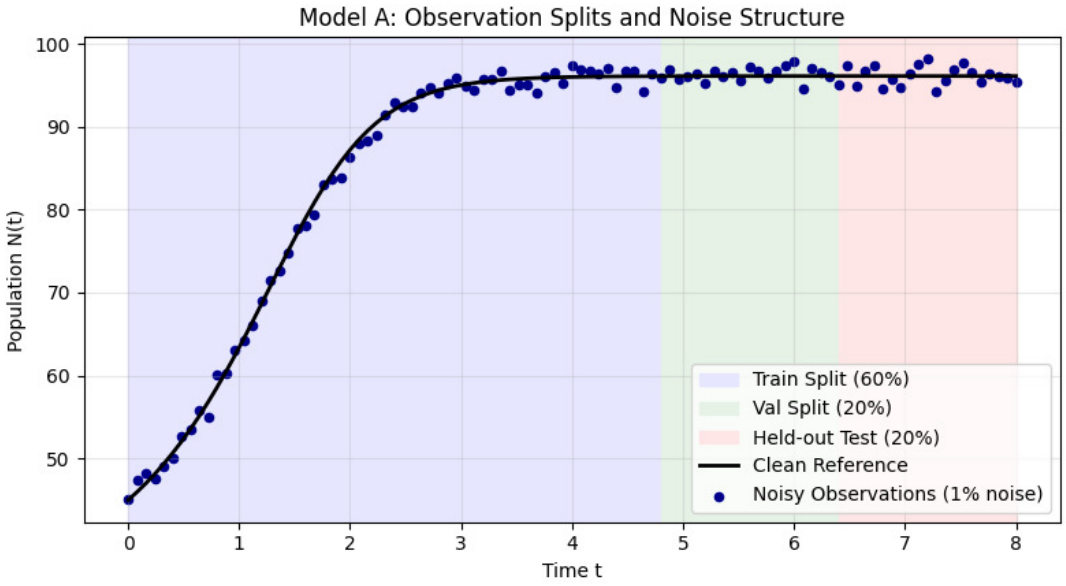


Figure 6.2: Model A observation design for the 1% noisy demonstration. The clean reference and noisy observations are shown with the contiguous training, validation, and held-out test windows.

For these demonstrations, the carrying capacity and initial population are treated as exact known conditions ($K = 100$ and $N(0) = 45$). The fitted physical parameters are therefore (r, A, H). The exact initial population is supplied to both methods, including for noisy observations, so both fits have a noise-free initial-condition anchor. This is an idealisation relative to a setting in which the initial population must itself be estimated.

6.3 Baseline inverse PINN on clean data

The first inverse-PINN demonstration uses the noise-free Model A observations ($\sigma = 0$). The true parameter values are $(r, A, H) = (0.70, 20.0, 10.0)$, and the common initial guesses are $(1.0, 30.0, 5.0)$. The network has two hidden layers of 32 units with tanh activation, and the initial condition is imposed exactly as $x(0) = x_0$.

The physical parameters are represented through smooth transformations that keep the estimates in their admissible ranges:

$$r = \text{softplus}(\widetilde{r}) > 0, \qquad A = K\ \text{sigmoid}(\widetilde{A}) \in (0, K), \qquad H = \text{softplus}(\widetilde{H}) \geq 0. \tag{6.9}$$

The objective combines data fidelity on the training observations ($t \leq 4.8$) with the Model A physics residual at 300 collocation points. The data and physics weights are $\lambda_{\text{data}} = 1.0$ and $\lambda_{\text{phys}} = 0.5$. The network learning rate is 10^{-3} and the physical-parameter learning rate is 5×10^{-3}. Training uses up to 500 Adam epochs followed by at most 3000 L-BFGS iterations. The run uses seed 1011.

The trainer returned status `converged` after 12.4 seconds, while L-BFGS itself terminated at its iteration limit rather than at a tolerance. The final estimates are:

Parameter	True value	PINN estimate	Relative error
r	0.7000	0.7002	0.03%
A	20.0000	20.0032	0.02%
H	10.0000	10.0025	0.03%

Thus, for this well-specified, noise-free single-case demonstration, the inverse PINN recovers all three parameters to within 0.03%. The notebook also records the parameter estimates throughout both the Adam and L-BFGS phases (Figure 6.3).

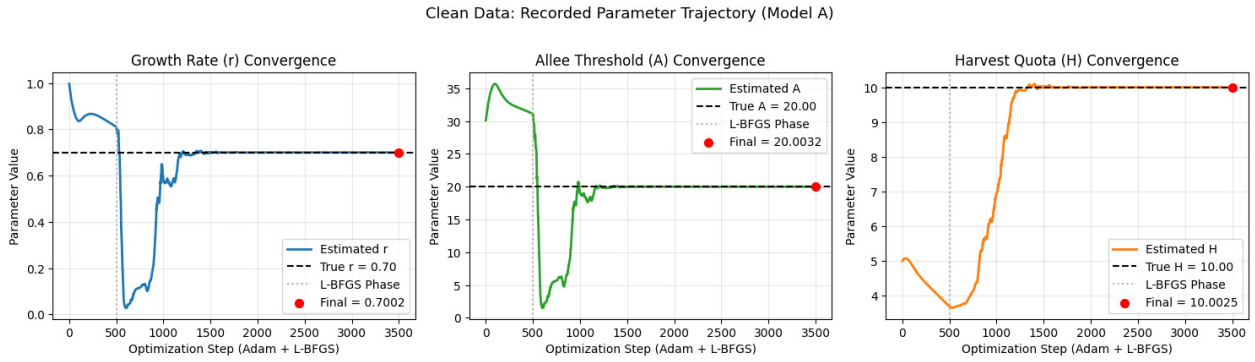


Figure 6.3: Recorded clean-data parameter trajectory for the inverse PINN. The three panels show the estimates of r, A, and H across the Adam exploration and L-BFGS refinement phases, together with their true values.

6.4 Parameter estimation under 1% measurement noise: ODE least squares versus PINN

The second live demonstration repeats the Model A fit using the same observations with $\sigma = 0.01$, corresponding to a noise standard deviation of 1.0. Both methods use the same training observations, known conditions ($K = 100$, $N(0) = 45$), and initial parameter guesses $(r, A, H) = (1.0, 30.0, 5.0)$.

The conventional `ODE_LS` baseline integrates the candidate ODE trajectory with an adaptive RK45 solver and estimates the three parameters using SciPy's Trust Region Reflective nonlinear least-squares algorithm. The inverse PINN jointly optimises the neural-network weights and physical parameters using the two-phase Adam–L-BFGS procedure. It uses the same two-hidden-layer, 32-unit, tanh network and 300 collocation points as the clean demonstration. For this noisy run, validation-based checkpoint selection is enabled. The network and parameter learning rates are 10^{-3} and 5×10^{-3}, respectively; the physics weight is 0.5; the Adam budget is up to 1200 epochs; the L-BFGS budget is 3000 iterations. The PINN uses seeds 10,11.

Table 6.1: Model A parameter estimates for the 1% noisy demonstration. The relative error is measured against the true parameter value.

Parameter	**True**	**ODE_LS estimate**	**ODE_LS error**	**PINN estimate**	**PINN error**
r	0.7000	0.5834	16.66%	0.5804	17.08%
A	20.0000	18.1335	9.33%	18.0672	9.66%
H	10.0000	9.1778	8.22%	9.1839	8.16%

The ODE fit takes 0.426 seconds, compared with 9.95 seconds for the PINN in this run. The PINN's L-BFGS phase reaches its iteration limit; validation selection is enabled for this noisy demonstration. The deviation from the true parameters can be understood from the nature of the inverse problem. Both methods minimize the discrepancy between the model and a finite set of noisy observations, rather than directly minimizing the error with respect to the true parameters. The injected noise has a standard deviation of 1 population unit, and only the first 61 of the 101 observations (up to $t = 4.8$) are used for training. Consequently, the particular realization of measurement noise (seed 11) influences the optimal parameter values: the noise deviations do not cancel out perfectly over this finite training sample, and the best-fitting parameters need not coincide with the values used to generate the data.

This effect is compounded by the interaction between the three unknown parameters in Model A,

$$\frac{dN}{dt} = rN\left(1 - \frac{N}{K}\right)\left(\frac{N}{A} - 1\right) - H.$$

Over the observed interval, changes in these parameters can partially compensate for one another. In particular, reducing r weakens population growth, whereas reducing A increases the Allee growth factor for $N > A$, and reducing H decreases the harvesting pressure. These competing effects allow different parameter combinations to produce very similar trajectories. The optimizers therefore settle on parameter values that provide a good fit to the particular noisy sample, rather than necessarily recovering the original parameters.

The close agreement between ODE_LS and the PINN is consistent with both methods finding a similar compromise under the same observations and governing equation. The PINN additionally optimizes the neural network weights alongside the physical parameters, and its L-BFGS phase reaches the iteration limit rather than satisfying a convergence tolerance. The held-out RMSE of approximately 1.09, which is close to the injected noise standard deviation, further suggests that both methods capture the underlying population trajectory to approximately the accuracy permitted by the noisy measurements. Thus, accurate trajectory prediction does not necessarily imply equally accurate parameter recovery. These results are one illustrative noisy fit, not a replacement for the multi-case benchmark in the next subsection.

6.5 Final sweep benchmark: 108 fits

The full benchmark contains 108 fits: 54 inverse-PINN fits and 54 `ODE_LS` fits. It spans six dynamical regimes, three noise levels, and three random seeds:

- **Model A:** `a_regular` (strong-Allee persistence), `a_near_fold` (near the saddle-node tipping point), and `a_extinction` (collapse below the Allee threshold).
- **Model B:** `b_regular` (proportional-harvest persistence), `b_near_fold` (near threshold), and `b_decline` (steep transient decline).
- **Noise levels:** $\sigma \in \{0, 0.01, 0.05\}$, with additive Gaussian noise standard deviation σN_{ref}.
- **Seeds:** 11, 22, and 33 for each case and noise level.

For clean data ($\sigma = 0$), a fit is counted as recovered when *every* fitted physical parameter is within 5% relative error of its true value. The reported median error is the median across the three seeds of each run's worst relative parameter error. The results are:

Table 6.2: Clean-data benchmark across the three seeds for each dynamical regime. PINN recovery counts use the 5% all-parameters criterion; median errors are the median worst-parameter relative error across seeds.

Dynamical case	PINN recovered	PINN median error	ODE_LS median error
`a_regular`	3/3	3.7×10^{-5}	2.9×10^{-8}
`b_regular`	3/3	2.7×10^{-5}	1.2×10^{-8}
`a_extinction`	1/3	1.3×10^{-1}	2.2×10^{-7}
`b_near_fold`	3/3	1.0×10^{-3}	4.7×10^{-8}
`a_near_fold`	0/3	1.0×10^{0}	7.0×10^{-8}
`b_decline`	0/3	2.9×10^{-1}	5.0×10^{-5}
Total	**10/18**		

On the clean benchmark, `ODE_LS` recovers all 18 of 18 fits within the 5% criterion, whereas the PINN recovers 10 of 18. The PINN succeeds on all three seeds for `a_regular`, `b_regular`, and `b_near_fold`; it succeeds on one of three `a_extinction` runs and on none of the `a_near_fold` or `b_decline` runs. A diagnostic analysis of why this happens is covered in the next subsection.

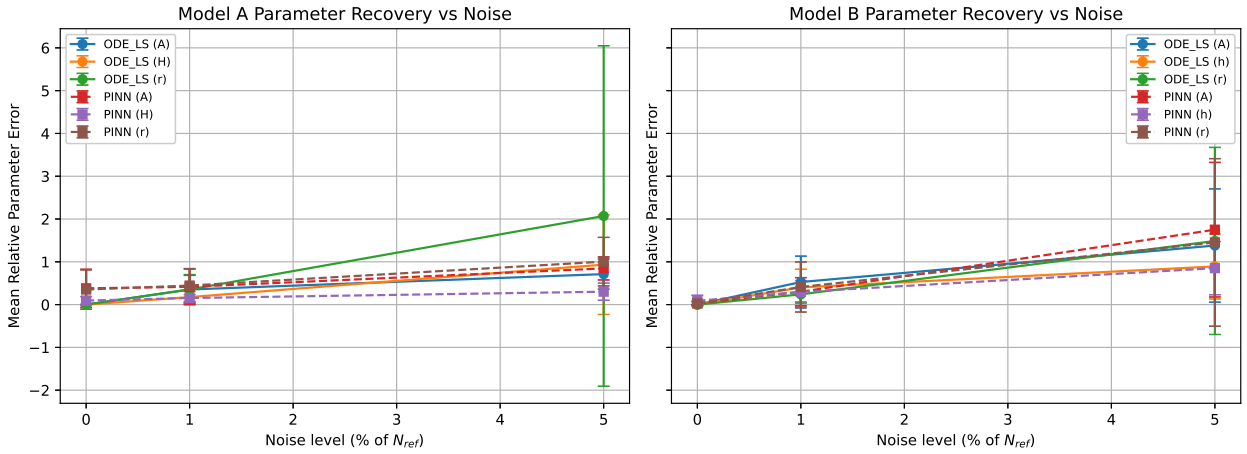


Figure 6.4: Mean relative parameter error versus noise level for the ODE least-squares fit and the PINN, for Models A and B. The plotted averages cover the six dynamical cases and three seeds; the error bars are strongly influenced by the poorly conditioned cases.

The noise plots above show consistent increase in parameter-recovery error as measurement noise increases from 0% to 5% for both models and optimisation methods. This is expected because noise obscures the parameter-dependent features of the observed trajectories, making it harder to distinguish between competing parameter combinations. Notably, non-zero errors are already present in some noise-free cases, particularly for the PINN in Model A, indicating that measurement noise is not the only source of error; the conditioning of the underlying inverse problem and optimisation difficulties also play a role. The widening error bars at higher noise levels further indicate that recovery becomes increasingly sensitive to the dynamical regime and random seed. Neither method consistently outperforms the other across all parameters, suggesting that the difficulty of the underlying inverse problem is an important factor in recovery accuracy.

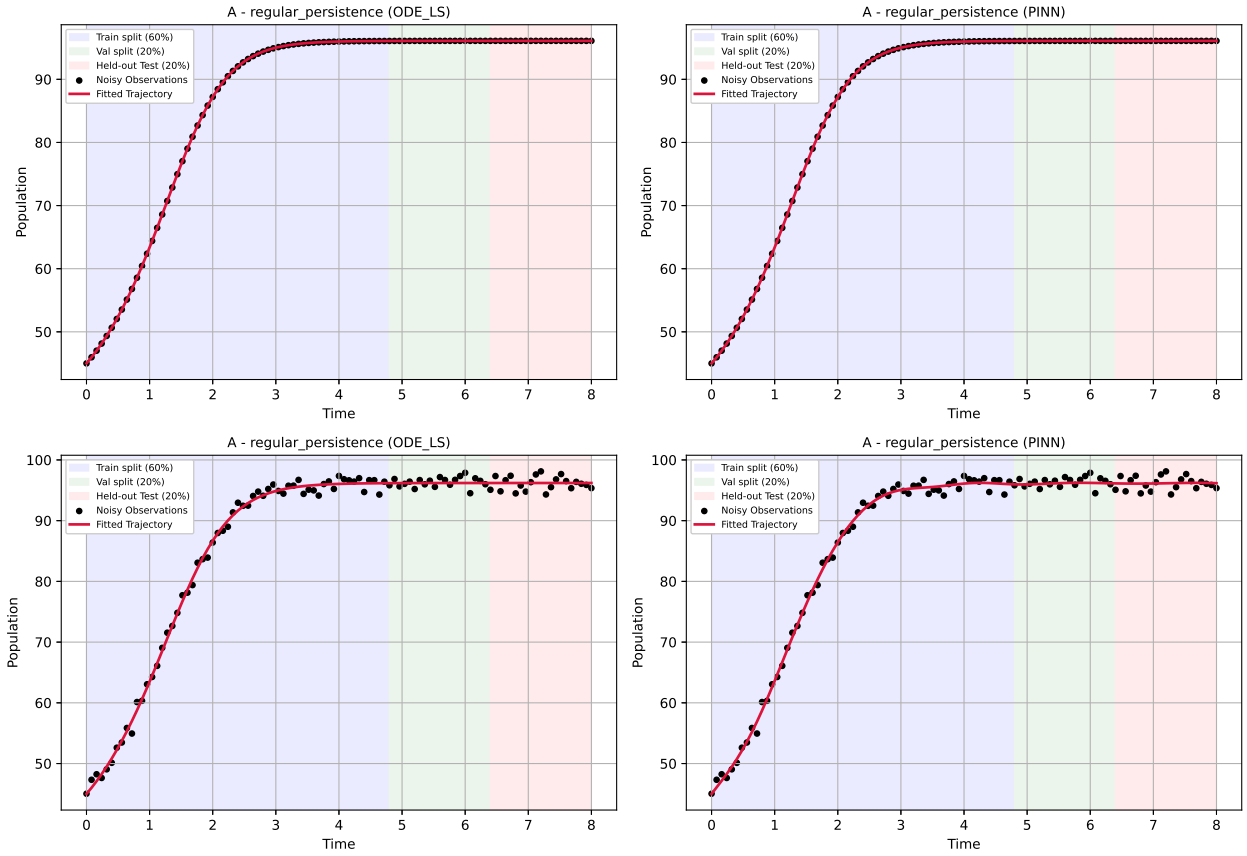


Figure 6.5: Model A regular fits with training, validation, and held-out windows. The top row is noise-free and the bottom row uses noisy observations. The held-out interval lies inside the PINN collocation domain.

Both methods reproduce the underlying trajectory closely, even when the observations contain 1% noise. In the noisy case, the fitted trajectories remain smooth rather than following individual fluctuations in the observations, indicating that the mechanistic ODE fit and physics constraint help capture the underlying population dynamics rather than measurement noise.

6.6 Diagnostic analysis: why eight PINN fits were not recovered

Across the 18 clean-data benchmark fits, `ODE_LS` meets the 5% recovery criterion in all 18, while the PINN meets it in 10. This difference is not explained by conditioning alone: the PINN recovers all three `b_near_fold` runs despite their relatively large reported condition number, but does not recover any of the `a_near_fold` or `b_decline` runs.

Recovered cases. The six `a_regular` and `b_regular` fits are all recovered. Their trajectories show smooth, monotonic sigmoidal growth, and the reported Jacobian condition numbers are in the range $\kappa(J) \in [62, 340]$. In these cases the effects of the physical parameters are sufficiently distinct over the transient phase for the data and physics objectives to support parameter recovery.

The `b_near_fold` case is recovered in all three seeds despite a reported condition number of approximately 10^3. A large condition number indicates an ill-conditioned sensitivity problem, not mathematical nonidentifiability by itself. In Model B, proportional harvesting hN scales with population and factors out N, while the trajectory in this case retains dynamic curvature

over time. L-BFGS refinement may help navigate poorly conditioned directions, but the experiment did not measure an exact Hessian or isolate the cause of the successful recovery.

Unrecovered cases. None of the three `a_near_fold` fits meets the recovery criterion. Near the saddle-node fold, fixed-quota harvesting creates a long dynamic bottleneck in which $dN/dt \approx 0$ (the "ghost" of the fold). Parameter sensitivities can become strongly correlated; in the recorded fits, r and A are nearly collinear, with r/A approximately constant. These fits can have small trajectory errors while their parameter estimates remain far from the true values. The bottleneck and correlated sensitivities are plausible explanations; gradient magnitudes were not measured in this experiment.

The `b_decline` case is not recovered in any of the three seeds, and `a_extinction` is recovered in only one of three. Both regimes include a steep, rapid transient collapse followed by a long, nearly flat tail near zero population. Limited information in the transient and the resolution of the collocation grid are possible contributors, but their causal effects on these inverse failures were not isolated by the study.

Why the ODE least-squares baseline performs better in this benchmark. The two optimisers solve substantially different numerical problems:

- **Search space.** `ODE_LS` estimates three physical parameters, $\boldsymbol{\theta} = (r, A, H)$ for Model A (or (r, A, h) for Model B). The PINN additionally optimises more than 1,000 neural-network weights and biases alongside the physical parameters.
- **Physics enforcement.** `ODE_LS` evaluates each candidate parameter set by numerically integrating the ODE with an adaptive RK45 solver and declared tolerances. The numerical solver has finite error; it is not exact arithmetic. The PINN enforces the ODE through a soft residual penalty evaluated at a finite set of collocation points.
- **Optimisation landscape.** The baseline uses bounded nonlinear least squares with the Trust Region Reflective algorithm. These results do not establish that its objective is convex or that it has a unique global optimum. The PINN instead optimises a composite data-and-physics objective over both network and physical parameters; such a problem can contain flat regions, saddle points, and competing minima.
- **Sensitivity calculation.** The ODE least-squares implementation estimates the residual Jacobian numerically and does not supply analytical sensitivities to the optimiser. The PINN uses gradients through the neural network and its composite loss.

The central distinction is that **parameter error is not trajectory error**. A PINN can fit observed points with a trajectory residual around 10^{-4} while its physical parameter estimates remain tens of per cent or, in some cases, approximately 100% away from the truth. The ODE baseline avoids this particular surrogate-fitting route by evaluating the forward ODE model directly for each candidate physical parameter set. Possible gradient-stiffness or spectral-bias effects—for example, network weights fitting observations faster than the physical parameters can improve the physics residual—are candidate explanations, not mechanisms demonstrated by the recorded benchmark.

Candidate follow-up methods. The following approaches are proposed experiments, not fixes demonstrated for the eight unrecovered PINN fits:

1. **Adaptive loss weighting:** methods such as SoftAdapt or GradNorm could dynamically balance the data and physics objectives rather than keeping their weights fixed.

2. **Residual-based adaptive refinement (RAR):** collocation points could be concentrated in steep collapse and inflection regions instead of remaining uniformly distributed in time.

3. **Curriculum training or time-domain decomposition:** training could progress through successive time windows, resolving the early transient before fitting later, flatter dynamics.

The scope of these diagnostics is limited. The initial population is independently known and exact, including in noisy runs. The ODE observation model uses zero padding after a candidate trajectory reaches extinction. The mechanistic explanations for the PINN failures remain hypotheses where the notebook does not report a direct measurement. The numerical methods used in the comparison are documented in the SciPy references for `least_squares` and L-BFGS-B.

6.7 Key scientific takeaways

1. **Parameter error is not trajectory error.** A PINN can fit observations to a trajectory residual of approximately 10^{-4} while its parameter estimates remain substantially inaccurate in ill-conditioned regimes.

2. **Classical ODE baselines are strong.** For these low-dimensional scalar ODEs, SciPy nonlinear least squares is faster and more robust in the clean-data benchmark, recovering the parameters across all six regimes.

3. **Structural identifiability precedes optimisation.** Model B with all four parameters (r, A, K, h) unknown is structurally ambiguous at a single fixed effort. No optimisation algorithm can resolve this ambiguity without independent information, such as a known carrying capacity K.

4. **The noise floor is a convergence diagnostic, not a parameter-accuracy guarantee.** Comparing training RMSE with the noise standard deviation, $\mathrm{RMSE}/(\sigma N_{\mathrm{ref}}) \approx 0.9$, measures the fit relative to the noise scale. It does not by itself establish parameter accuracy or show that all available information has been extracted.

7 Conclusions and limitations

7.1 What we found

Analytical results. For a stock fished with a fixed quota H, a strong Allee effect creates a pair of steady states, an unstable threshold and a stable upper state, which approach each other as the quota rises and merge at the critical catch $H_c = rKq_c$ (Section 2). As the quota approaches this limit, the gap between them, which is the stock's safety margin, shrinks like $\sqrt{q_c - q}$, so a stock fished close to the limit can be tipped into collapse by a very small shock. A fixed fraction of the stock (Model B) has an analogous critical effort $h_c = r(1-a)^2/(4a)$ (Section 3). The two rules must be compared in common units and with a stated calibration: with the same first-year catch the logistic stock collapsed in our example while both Allee

stocks survived, but with equal fractions of their own thresholds all three survived, so no universal ranking follows.

Collapse and recovery. Reducing harvesting after a collapse does not necessarily restore the population. Once it has fallen below the Allee threshold, natural growth remains negative even without harvesting. Recovery may therefore require an intervention such as restocking the population above the relevant recovery threshold (Section 3). Section 4 also showed how the initial population determines survival, how extinction times can be calculated, and why collapse may be delayed when constant harvesting is just above its critical rate.

Forward PINN. A small tanh network with the initial condition built in and a physics-only loss reproduced the solution of all six test cases to a relative L^2 error between 2.9×10^{-7} and 4.1×10^{-3} (36 of 36 runs accurate over two sets of seeds). It was much slower than a standard solver, which reaches about 10^{-10} in a few milliseconds, so for a single scalar equation the PINN is not the better tool (Section 5).

Inverse PINN. Using one shared setting for the forward and inverse problems, the PINN recovered every fitted parameter within 5% in 10 of 18 clean fits, against 18 of 18 for ordinary least squares and 0 of 18 for the settings first tried (Section 6). Under noise, recovery was limited by the data themselves. Noise was not the reason some fits failed; the fits that did not converge came from validation-based checkpoint selection, and the remaining recovery failures were the poorly conditioned cases. The one provably impossible case, Model B with all four parameters unknown, was kept separate from these.

7.2 Lessons about method

- A small training loss is not evidence of accuracy. The early forward PINNs had a small loss at the collocation points and a large error between them.
- A good trajectory fit does not guarantee good parameters. The inverse PINN fitted every training set to about 10^{-4} and still got some parameters wrong by 100%.
- Check convergence against what is achievable. Comparing the training error with the noise level exposed runs that had stopped without learning.
- A classical baseline should always be run. Here it was faster and more accurate than the PINN on every clean case.

7.3 Limitations

- All data are synthetic and generated from the same models that are fitted, so there is no model error and nothing here validates the models for a real species.
- The models are deterministic and unstructured. Random fluctuations, age structure, delays, spatial effects and changing parameters can all matter most near a threshold.
- The PINN studies use one parameter set per case and three seeds per setting. Differences between settings are therefore not statistically tested, and the choice of the informative-case threshold was made after seeing the results.

- In the inverse study K and the starting population are assumed known and exact, which makes the problem easier than a field setting.
- The iteration budget of the L-BFGS phase was binding in every PINN run, and longer runs were not tried.
- Results depend on the platform and library versions (recorded in the run manifests), and the long sweeps are recorded evidence rather than re-run for this report.

7.4 Future work

Natural next steps are to add random fluctuations and age structure to the model, to test other parameter values and more observations in the inverse problem, to try bounds or a different parametrisation to stop r and A collapsing together in the near-fold cases, and, if real data become available, to fit a real fishery with its provenance documented.

8 Contributions and Acknowledgements

Contributions. The project was divided into five main parts. Raghav Singhal worked on the mathematical foundations and Model A, Ashwath Chandramouli on Model B and the comparison of harvesting methods, and Devam Jatin Bhavsar on collapse thresholds, extinction times and numerical verification. Mehta Prem worked on the forward PINN analysis, while Rohan Agrawal worked on inverse PINNs, parameter estimation and validation. The notebook, report and presentation were prepared and integrated as a group.

AI assistance. We used ChatGPT, OpenAI Codex, Claude Code and Antigravity AI Assistant during the project. ChatGPT assisted with mathematical explanations, review and editing of written material. OpenAI Codex was used for code development, debugging, numerical checks, testing and project integration. Claude Code and Antigravity AI Assistant supported implementation, numerical experiments and preparation of code and written drafts, including work related to the PINNs.

The AI-generated suggestions and outputs were treated as supporting material. The mathematical results, numerical findings and submitted explanations remain the responsibility of the authors.

Sources and Data. All population trajectories, reference solutions and noisy observations used in this project were generated synthetically from the mathematical models described in this report. No external population dataset was used. External mathematical and computational sources are cited where relevant.

The complete source code, experiment configurations, numerical data, saved results and figures are available in the GitHub project repository. The accompanying Jupyter notebook reproduces the main analytical and numerical calculations, executes short PINN demonstrations and loads the recorded results of the larger PINN experiments.

References

[1] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-

Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs. `http://github.com/jax-ml/jax`, 2018.

[2] Fred Brauer and Carlos Castillo-Chavez. *Mathematical Models in Population Biology and Epidemiology*, volume 40 of *Texts in Applied Mathematics*. Springer, New York, 2nd edition, 2012.

[3] Franck Courchamp, Luděk Berec, and Joanna Gascoigne. *Allee Effects in Ecology and Conservation*. Oxford University Press, Oxford, 2008.

[4] Brian Dennis. Allee effects: population growth, critical density, and the chance of extinction. *Natural Resource Modeling*, 3(4):481–538, 1989.

[5] J. R. Dormand and P. J. Prince. A family of embedded Runge–Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980.

[6] E. Hairer, S. P. Nørsett, and G. Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*, volume 8 of *Springer Series in Computational Mathematics*. Springer, Berlin, 2nd revised edition, 1993.

[7] Jeffrey A. Hutchings and John D. Reynolds. Marine fish population collapses: consequences for recovery and extinction risk. *BioScience*, 54(4):297–309, 2004.

[8] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, San Diego, 2015.

[9] I. E. Lagaris, A. Likas, and D. I. Fotiadis. Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks*, 9(5):987–1000, 1998.

[10] Dong C. Liu and Jorge Nocedal. On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, 45(1–3):503–528, 1989.

[11] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[12] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for activation functions. arXiv:1710.05941, 2017.

[13] SciPy developers. quad documentation. `https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.quad.html`. Accessed 28 October 2026.

[14] SciPy developers. solve_ivp documentation. `https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html`. Accessed 19 October 2026.

[15] Philip A. Stephens and William J. Sutherland. Consequences of the Allee effect for behaviour, ecology and conservation. *Trends in Ecology & Evolution*, 14(10):401–405, 1999.

[16] Steven H. Strogatz. *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*. Westview Press, Boulder, CO, 2nd edition, 2015.